

Relatório do Projeto 2

Link do GitHub:

<https://github.com/jhapreis/7600017-introducao-fisica-computacional/tree/main/projeto1>

Sumário

1	Tarefa 1 - Números aleatórios, integrais e amostragem	1
1.1	Gerador de sequências pseudo-aleatórias	1
1.2	Histogramas dos valores médio e desvio padrão	3
1.3	Estimativa do valor de π com integração por Monte Carlo	5
1.3.1	Motivação do problema	5
1.3.2	Ajuste dos intervalos da função <i>rand</i>	6
1.3.3	Estimativa de N_{total}	6
1.3.4	Programa com sequências pseudo-aleatórias e resultados	7
1.4	Cálculo de integrais por Monte Carlo	8
1.4.1	Motivação com o Teorema do Valor Médio	8
1.4.2	Estimativa de N_{total}	8
1.4.3	Cálculo das integrais	8
2	Tarefa 2 (Opcional) - Decaimento Radioativo	10
3	Tarefa 3 - Passeio Aleatório	11
3.1	Passeio aleatório em 1D	11
3.1.1	Cálculo do valor teórico	11
3.1.2	Resultado das simulações	11
3.2	Passeio aleatório em 2D	14
3.2.1	Cálculo do valor teórico	14
3.2.2	Resultado das simulações	15
3.3	(Opcional) Cálculo da Entropia e Seta do Tempo	16

1 Tarefa 1 - Números aleatórios, integrais e amostragem

1.1 Gerador de sequências pseudo-aleatórias

Na figura 1 podemos observar o código utilizado para gerar as sequências pseudo-aleatórias.

A saída do programa são arquivos de dados contendo, respectivamente:

- O identificador da sequência
- O tamanho da sequência
- A média $\langle r \rangle$ dos valores da sequência
- O desvio padrão σ_r dos valores da sequência

```
character file_number

integer iseed, m, n_size, k_seq

double precision r, r2, sum_r, sum_r2
double precision mean_r, mean_r2, std

iseed = 50

call srand(iseed)

do m=1,9 ! com m=1, 2, 3, ..., 9

    file_number = char(m+48) ! Transforma o número em caracter pela tabela ASCII ("0" = 48)

    print *, file_number

    open(unit=10, file="arquivo-saida-//file_number//".dat")

    do k_seq=1,100 ! gere 100 sequências

        sum_r = 0.0d0
        sum_r2 = 0.0d0

        do n_size=1,3**m - 1 ! de tamanho 3**m
            r = rand()
            r2 = r**2
            sum_r = sum_r + r
            sum_r2 = sum_r2 + r2
        end do

        mean_r = sum_r / real(n_size, 8)
        mean_r2 = sum_r2 / real(n_size, 8)
        std = sqrt(mean_r2 - mean_r**2)

        write(10, *) k_seq, n_size, mean_r, std

    end do

    close(10)

end do

stop
end
```

Figura 1: Código para geração das $k = 10^2$ sequências pseudo-aleatórias, com tamanhos $N = 3^m$, onde $m = 1, 2, 3, \dots, 9$

Graficando os valores das médias $\langle r \rangle$ e dos desvios-padrão σ_r para cada sequência, em função do tamanho N , podemos notar que conforme N aumenta os valores se dispersam menos e se aproximam cada vez mais dos valores esperados:

$$\langle r \rangle = \frac{1}{2} \quad (1)$$

$$\sigma_r = \frac{1}{2\sqrt{3}} \approx 0,288675 \quad (2)$$

As figuras 2 e 3 mostram os resultados.

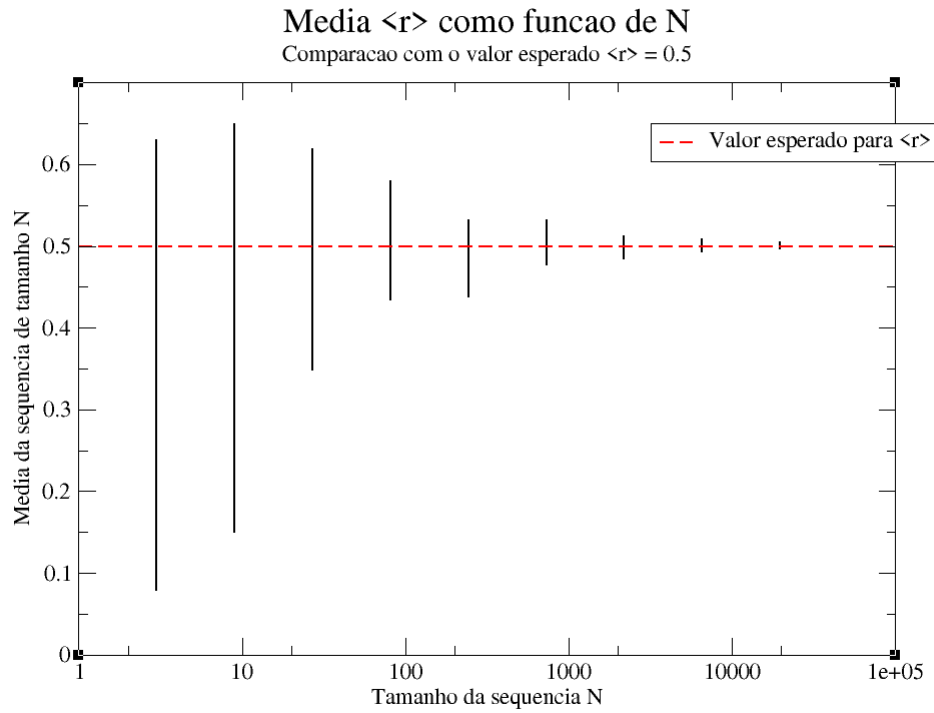


Figura 2: Gráfico das médias $\langle r \rangle$ das sequências em função da quantidade de sequências N .

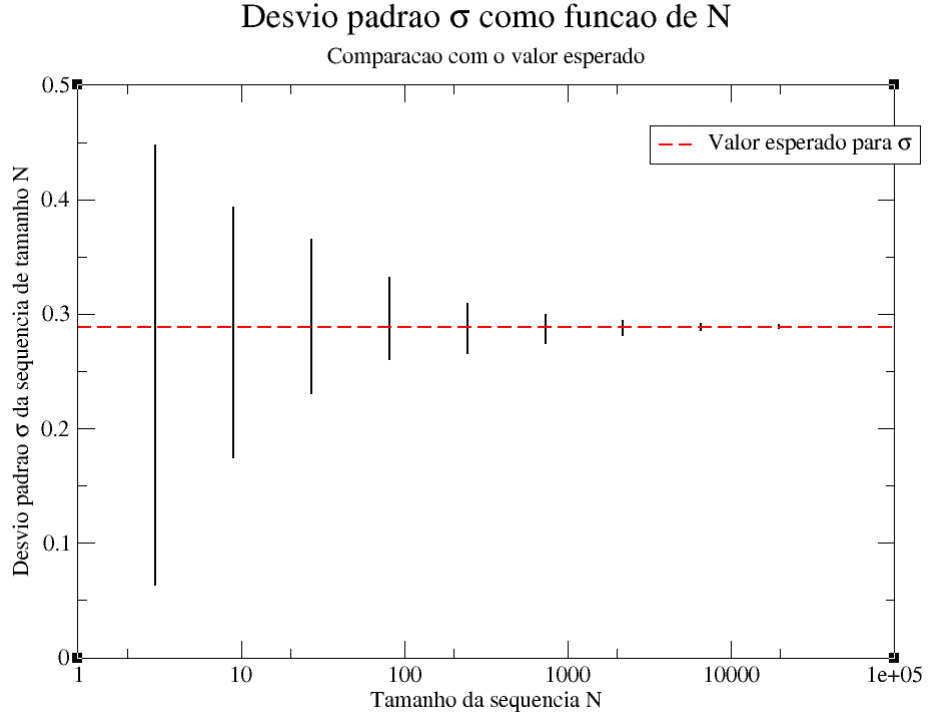


Figura 3: Gráfico dos desvios-padrão σ_r das sequências em função da quantidade de sequências N .

1.2 Histogramas dos valores médio e desvio padrão

Pelo Teorema do Limite Central, as gaussianas esperadas são dadas por

$$P(x) = \frac{1}{\sqrt{2\pi\sigma_{\langle r \rangle}^2}} \cdot \exp\left(-\frac{(x - \mu)^2}{2\sigma_{\langle r \rangle}^2}\right) \quad (3)$$

Como visto anteriormente, conhecemos os valores teóricos de

$$\langle r \rangle = \frac{1}{2}$$

e de

$$\sigma_{\langle r \rangle} = \frac{\sigma_r}{\sqrt{N}} = \frac{1}{2\sqrt{3N}}$$

Considerando $N = 3^2, 3^3, 3^4$, obtemos que

$$\begin{aligned} P_{N=9}(x) &= \sqrt{\frac{54}{\pi}} \exp(-54(x - 0,5)^2) \\ P_{N=27}(x) &= \sqrt{\frac{162}{\pi}} \exp(-162(x - 0,5)^2) \\ P_{N=81}(x) &= \sqrt{\frac{486}{\pi}} \exp(-486(x - 0,5)^2) \end{aligned}$$

Nas figuras 4 e 5 podemos observar o comportamento das distribuições ao redor do valor esperado e também a dependência de $\sigma \sim \frac{1}{\sqrt{N}}$

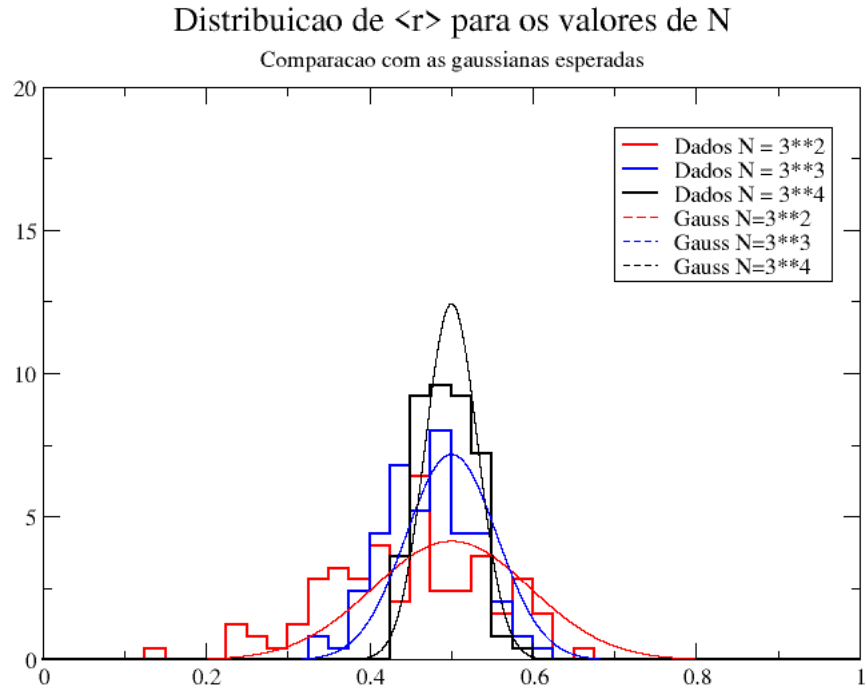


Figura 4: Gráfico dos histogramas normalizados dos valores médios das k sequências para $N = 3^2, 3^3, 3^4$, comparando com as suas respectivas gaussianas e com o valor teórico de $\langle r \rangle = \frac{1}{2}$

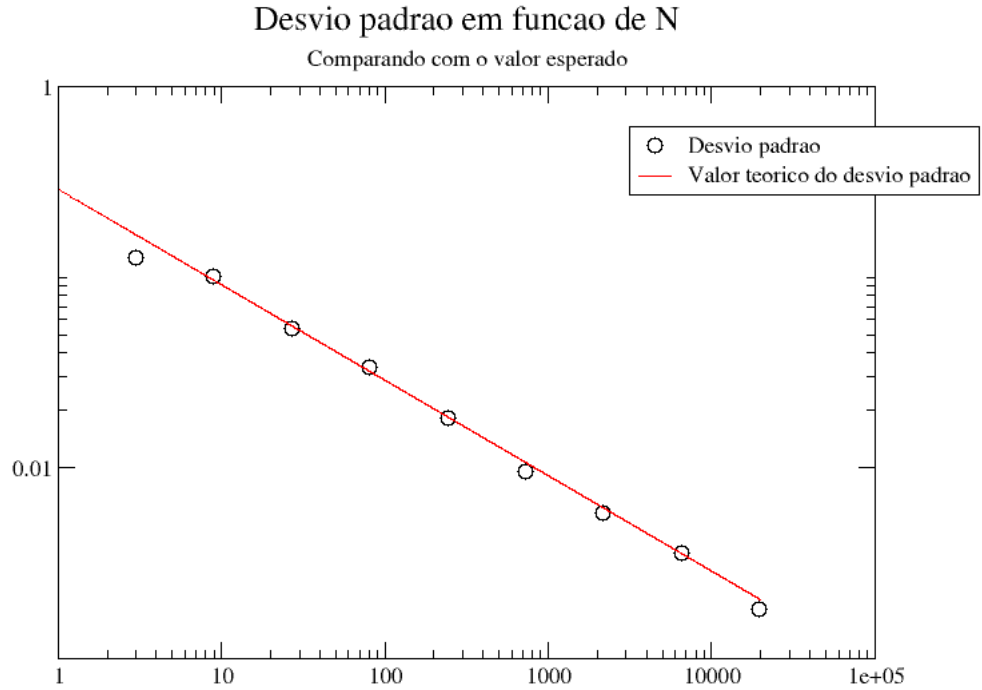


Figura 5: Gráfico log-log para o desvio padrão em função do tamanho da amostra, comparando com o valor teórico de $\sigma_{\langle r \rangle} = \frac{1}{2\sqrt{3N}}$

```

! Programa para calcular o desvio padrão das médias em cada arquivo do problema 1a
! Nesse contexto, agora r representa a média da sequência (não mais o valor em cada elemento da sequência)

implicit none

character(100) filename

integer k_seq, n, k, i
double precision r, std_r ! Variáveis para leitura do arquivo
double precision sum_r, sum_r2, mean_r, mean_r2, std

open(unit=10, file="arquivo-saida-mean-std.dat")

do i=1,9

    filename = "../tarefa-1a/arquivo-saida-//"char(i+48)//".dat"

    open(unit=20, file=filename)

    sum_r = 0.0d0
    sum_r2 = 0.0d0

    do k=1,100
        read(20, *) k_seq, n, r, std_r
        sum_r = sum_r + r
        sum_r2 = sum_r2 + r**2
    end do

    close(20)

    mean_r = sum_r / real(k_seq, 8)
    mean_r2 = sum_r2 / real(k_seq, 8)
    std = sqrt(mean_r2 - mean_r**2)

    write(10, *) n, std

end do

close(10)

stop
end

```

Figura 6: Código principal da tarefa 1b, para cálculo do desvio padrão das médias.

1.3 Estimativa do valor de π com integração por Monte Carlo

1.3.1 Motivação do problema

Dada uma função de probabilidade com distribuição uniforme, podemos considerar que, para N grande, a razão entre a quantidade de pontos dentro da curva e quantidade de pontos totais se aproxima da razão da área da curva com a área total.

Ou seja,

$$\frac{A_{circ}}{A_{total}} \approx \frac{N_{dentro}}{N_{total}}$$

sendo uma aproximação cada vez melhor conforme N_{total} é cada vez maior (considerando o Teorema do Limite Central).

No nosso caso, como

$$\frac{A_{circ}}{A_{total}} = \frac{\pi \cdot 1^2}{2 \cdot 2} = \frac{\pi}{4}$$

isso significa que

$$\frac{N_{dentro}}{N_{total}} \approx \frac{\pi}{4} \Rightarrow \pi \approx 4 \cdot \frac{N_{dentro}}{N_{total}}$$

Em resumo, obter o valor de π ao rodar um programa que nos dará N_{dentro} e N_{total} . Para cada ponto $P_i = (x_i, y_i)$ gerado, podemos saber se ele está dentro do círculo verificando se $x_i^2 + y_i^2 \leq 1$. A quantidade de pontos que obedecerem essa relação nos dará N_{dentro} , enquanto a contagem total nos dará N_{total} .

1.3.2 Ajuste dos intervalos da função *rand*

Considerando que o FORTRAN nos dá a função *rand()* no intervalo $[0, 1]$, podemos ajustar os valores conforme nossa necessidade para gerar números aleatórios no intervalo $[a, b]$ (com $a < b$):

$$f(x) = (b - a) \cdot x + a \quad (4)$$

Como $f(x)$ é uma transformação linear da variável x , e como a função de probabilidade associada a x é uniforme (a mesma probabilidade para todos os valores), segue que $f(x)$ também tem probabilidade uniforme, porém agora no intervalo $[a, b]$.

Para o nosso problema, precisamos gerar números aleatórios dentro do quadrado definido pelas coordenadas (x, y) onde $-1 \leq x, y \leq 1$. De onde segue que

$$f(x) = 2x - 1 \quad (5)$$

1.3.3 Estimativa de N_{total}

Para estimar N_{total} de modo a obtermos π com 4 casas decimais de precisão, podemos considerar que $\sigma < 0,00005 = 5 \times 10^{-5}$.

Como

$$\sigma \sim \frac{1}{\sqrt{N_{total}}} \quad (6)$$

de onde tiramos que

$$N_{total} > \frac{1}{(5 \times 10^{-5})^2} = 4 \times 10^8$$

Ou seja, $N_{total} > 4 \times 10^8$ para obtermos π com 4 casas decimais de precisão.

1.3.4 Programa com sequências pseudo-aleatórias e resultados

Representação da contagem de pontos dentro da circunferência

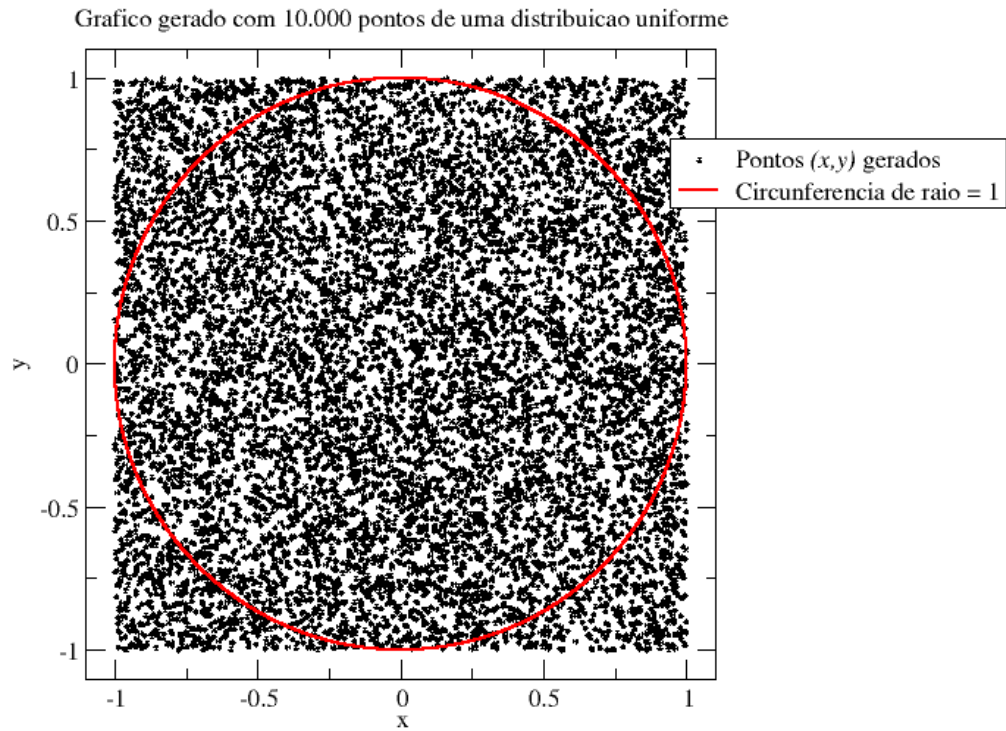


Figura 7: Resultado gráfico de 10.000 pontos gerados pela distribuição uniforme. Podemos observar que, conforme N_{total} cresce, o gráfico fica cada vez mais preenchido (aproximando-se da área exata).

No programa principal foram adicionados duas variáveis para servir como *flag*: uma para acompanhar a evolução do loop (para debug) e outra para salvar os dados num arquivo de destino (usado para gerar o plot da figura 7). Os resultados podem ser vistos na Tabela 1.

N_{total}	π	Erro
4×10^6	3,142418	$8,25 \cdot 10^{-4}$
4×10^8	3,141594	$1,16 \cdot 10^{-6}$
4×10^{10}	3,141593	$3,72 \cdot 10^{-7}$

Tabela 1: Tabela dos resultados variando o N_{total} , comparando com o valor conhecido de $\pi \approx 3,14159265359$.

Os resultados mostram uma grande problemática do método, que acaba também sendo uma de suas maiores virtudes: para melhorar a aproximação do resultado, nós precisamos saltar para ordens de N_{total} cada vez mais altas – porém esta é a única imposição necessária.

```

1  implicit none
2
3  ! Gerador de pares x,y para calcular o valor de pi
4  ! O intervalo é ajustado de [0,1] -> [-1,1] fazendo f(x) = 2x - 1
5
6  integer is_on_the_circle
7  double precision get_random_coordinate
8
9  integer iseed
10 double precision n_total, n_dentro, i
11 double precision x, y, pi
12
13 integer fl_debug, fl_save_to_file
14
15 fl_debug = 1
16 fl_save_to_file = 0
17
18 iseed = 50
19
20 open(unit=40, file="tarefa-1c-saida.dat")
21
22 n_total = 4.0d0
23
24 n_dentro = 0.0d0
25
26 i = 1.0d0
27
28 call srand(iseed)
29
30
31 do while(.le.n_total)
32   x = get_random_coordinate()
33   y = get_random_coordinate()
34
35   if(is_on_the_circle(x, y).eq.1) then
36     n_dentro = n_dentro + 1.0d0
37   end if
38
39   i = i + 1.0d0
40
41   ! Se for requisitado, exibe logs de debug
42   if(fl_debug.eq.1) then
43     if(mod(i, 1.0d0).eq.0) then
44       print *, "[DEBUG] Processando...", 1.0d2 * i / n_total, "%"
45     end if
46   end if
47
48   ! Se for requisitado, salva os dados para plot posterior
49   if(fl_save_to_file.eq.1) then
50     write(40, *) x, y
51   end if
52
53 end do
54
55 pi = 4.0d0 * n_dentro / n_total
56
57 print *, 'pi=', pi
58
59 close(40)
60
61 stop
62 end
63
64
65 double precision function get_random_coordinate()
66   get_random_coordinate = 2.0d0*rand() - 1.0d0
67 return
68 end
69
70 integer function is_on_the_circle(x, y)
71   double precision x, y
72
73   if((x**2 + y**2).le.1.0d0) then
74     is_on_the_circle = 1
75   else
76     is_on_the_circle = 0
77   end if
78 return
79 end

```

Figura 8: Código principal da tarefa 1c, para cálculo do valor de π .

1.4 Cálculo de integrais por Monte Carlo

1.4.1 Motivação com o Teorema do Valor Médio

O Teorema do Valor Médio relaciona o valor de uma integral ao valor médio $\langle f \rangle$ dentro de um intervalo $[a, b]$:

$$\int_a^b f(x)dx = (b - a)\langle f \rangle$$

onde

$$\langle f \rangle \approx \frac{1}{N} \sum_{i=1}^N f(x_i) \quad , \text{ converge para } N \rightarrow \infty$$

Ou seja, para nosso uso:

$$\int_a^b f(x)dx = \frac{(b - a)}{N} \sum_{i=1}^N f(x_i) \quad (7)$$

1.4.2 Estimativa de N_{total}

Semelhante ao que fizemos anteriormente: para 3 casas decimais de precisão, $\sigma < 0,0005 = 5 \times 10^{-4}$

$$\sigma \sim \frac{1}{\sqrt{N_{total}}} \Rightarrow N_{total} > \frac{1}{(5 \times 10^{-4})^2} = 4 \times 10^6$$

Ou seja, precisamos de $N_{total} > 4 \times 10^6$ para garantir 3 casas decimais de precisão.

1.4.3 Cálculo das integrais

$$I_i = \int_0^1 \frac{4}{1+x^2} dx = 4 \cdot [\arctan(x)]_0^1 = \pi \approx 3,141593$$

$$I_{ii} = \int_1^{10} \frac{1}{x} dx = [\ln(x)]_1^{10} = \ln(10) \approx 2,302585$$

Para $N_{total} = 4 \times 10^6$, o resultado é apresentado na Tabela 2. Os valores estão muito próximos ao valor da integral, mostrando a eficiência do método.

	Integral	Erro
I_i	3,141	$5,418 \cdot 10^{-4}$
I_{ii}	2,302	$8,020 \cdot 10^{-4}$

Tabela 2: Tabela dos resultados dos cálculos das integrais dos itens i e ii usando o Teorema do Valor Médio, com $N_{total} = 4 \times 10^6$

```

implicit none

integer iseed
double precision n_total, a1, b1, a2, b2
double precision f1, f2, integral, random
double precision i, x1, x2, sum_f1, sum_f2
double precision integral_f1, integral_f2

iseed = 50

call srand(iseed)

n_total = 4.0d6

a1 = 0.0d0
b1 = 1.0d0

a2 = 1.0d0
b2 = 10.0d0

i = 1.0d0
sum_f1 = 0.0d0
sum_f2 = 0.0d0

do while(i.le.n_total)
    x1 = random(a1, b1)
    sum_f1 = sum_f1 + f1(x1)

    x2 = random(a2, b2)
    sum_f2 = sum_f2 + f2(x2)

    i = i + 1.0d0
end do

integral_f1 = integral(sum_f1, a1, b1, n_total)
integral_f2 = integral(sum_f2, a2, b2, n_total)

print *, "integral_f1=", integral_f1
print *, "integral_f2=", integral_f2

stop
end

```

```

! Função para gerar valores aleatórios de f(x) = 4 / (1 + x^2), 0 <= x <= 1
double precision function f1(x)
    double precision x
    f1 = 4.0d0 / (1.0d0 + x**2)
return
end

! Função para gerar valores aleatórios de f(x) = 1 / x, 1 <= x <= 10
double precision function f2(x)
    double precision x
    f2 = 1.0d0 / x
return
end

! Função para gerar valores aleatórios com probabilidade uniforme entre [a,b]
double precision function random(a, b)
    double precision a, b
    random = (b - a) * rand() + a
return
end

! Função para o cálculo da integral pelo Teorema do Valor Médio
double precision function integral(sum_fx, a, b, n_total)
    double precision sum_fx, a, b, n_total
    integral = (b - a) / n_total * sum_fx
return
end

```

Figura 9: Código principal da tarefa 1d, para cálculo das integrais usando o método de Monte Carlo.

2 Tarefa 2 (Opcional) - Decaimento Radioativo

3 Tarefa 3 - Passeio Aleatório

3.1 Passeio aleatório em 1D

3.1.1 Cálculo do valor teórico

Da Teoria da Probabilidade, sabemos que

$$\langle x \rangle = \int_{-\infty}^{\infty} x f(x) dx \quad (8)$$

onde $f(x)$ é a função de distribuição de probabilidade. No nosso caso, como a probabilidade é uniforme de $[-1, 1]$, temos que $f(x) = \frac{1}{2}$.

Ou seja,

$$\begin{aligned} \langle x \rangle &= \int_{-\infty}^{\infty} x f(x) dx = \int_{-1}^1 x \cdot \frac{1}{2} dx = 0 \\ \langle x^2 \rangle &= \int_{-1}^1 x^2 \cdot \frac{1}{2} dx = \frac{1}{3} \end{aligned}$$

3.1.2 Resultado das simulações

Como vimos na seção 1.2, podemos estimar o tamanho de k para obter um erro da ordem de 10% ao utilizar a equação 6:

$$k \approx \frac{1}{0,10^2} = 100$$

Ou seja, usaremos $k = 100$ como o número de sequências geradas, com o número de passos $t = 50, 100, 200$. Além disso, usaremos a mesma equação 5 para podera ajustar o valor gerado pelo FORTRAN.

Na figura 10 podemos observar as distribuições obtidas para os instantes de tempo $t = 50, 100, 200$. Nas figuras 11 e 12 podemos observar como $\langle x \rangle(t)$ e $\langle x^2 \rangle(t)$ se comportam em torno dos valores teóricos.

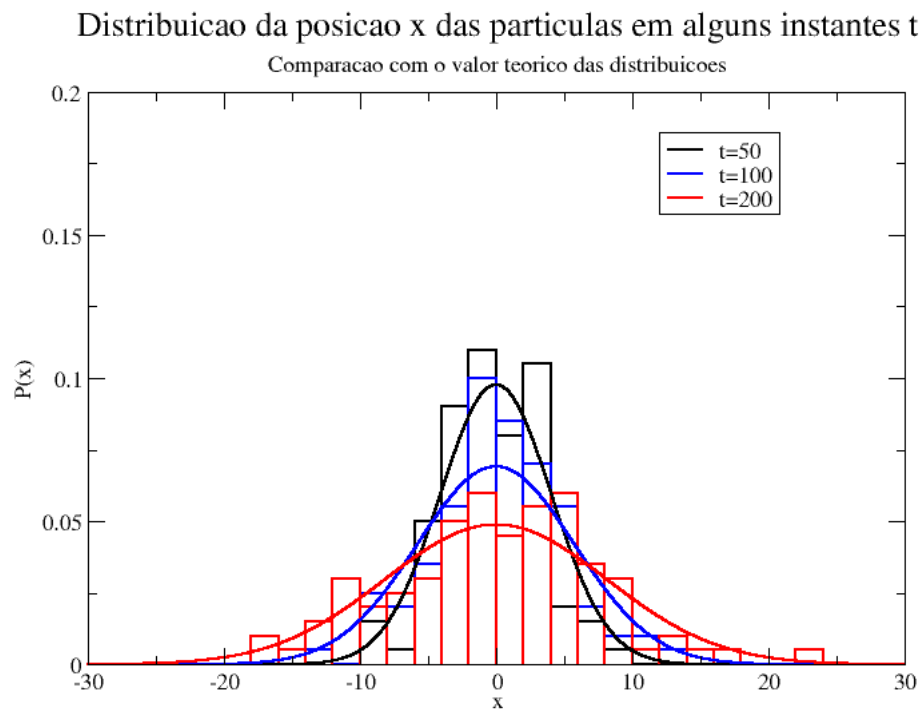


Figura 10: Distribuição da posição das partículas em $t = 50, 100, 200$. É possível observar que, conforme t aumenta, mais disperso fica o gráfico – mesmo que ele continue centrado na origem.

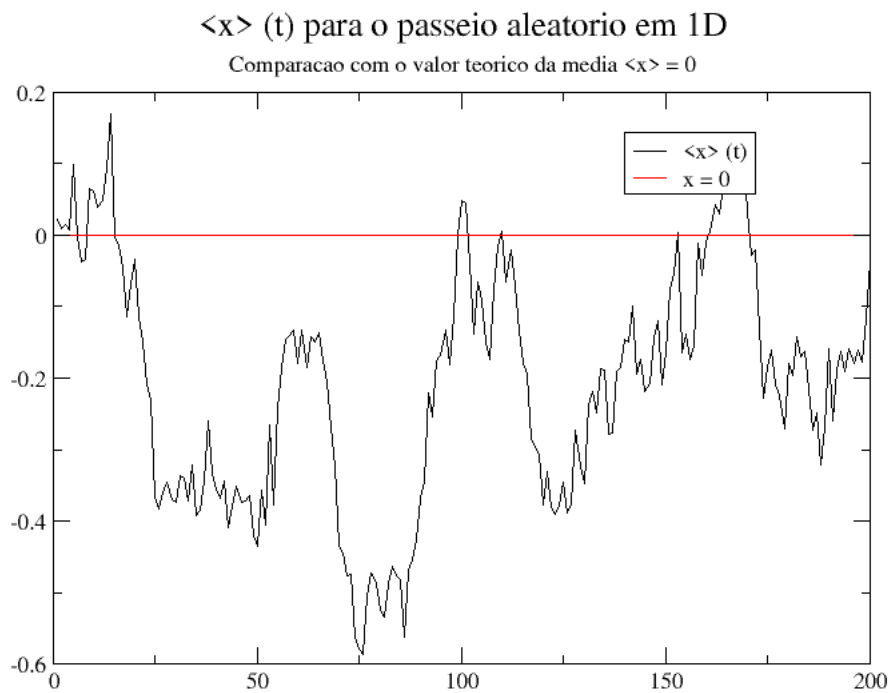


Figura 11: Valor de $\langle x \rangle(t)$. Observe como o valor fica oscilando em torno da origem.

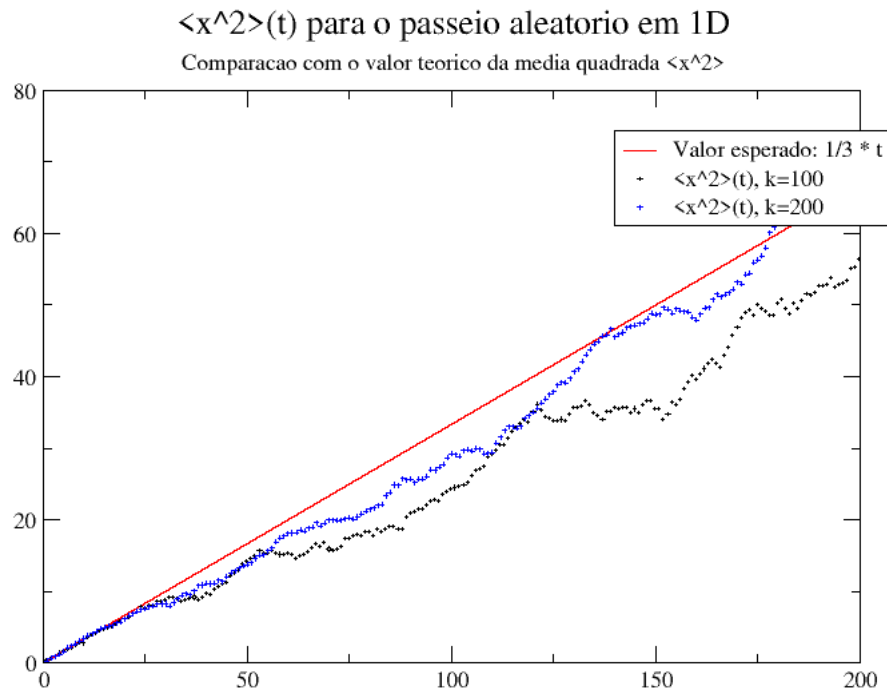


Figura 12: Valor de $\langle x^2 \rangle(t)$, em comparação com o valor esperado de $\langle x^2 \rangle(t) = \frac{t}{3}$. Observação: para tornar mais evidente a dependência linear com t , foi adicionado também o plot com $k = 200$ (além de $k = 100$).

```

1  implicit none
2
3  integer k_total ! Número de sequências ("passeios", ou "partículas a caminhar")
4  integer t_max  ! Tempo máximo para a simulação
5  integer iseed
6  integer k, t
7
8  double precision sum_x, sum_x2, mean_x, mean_x2
9  double precision pos_x(500) ! Lista com a posição em x de cada partícula. O valor é grande para poder variar k_total
10
11 double precision step
12
13 iseed = 50
14 k_total = 100
15 t_max = 200
16
17 call srand(iseed)
18
19 call null_vector(pos_x, k_total)
20
21 ! Arquivo de saída para o gráfico de t, <x(t)> e <x2(t)>
22 open(unit=10, file="arquivo-saida-media-posicao-tempo.dat")
23
24 ! Arquivos de saída para os histogramas
25 open(unit=50, file="arquivo-saida-posicoes-t50.dat")
26 open(unit=100, file="arquivo-saida-posicoes-t100.dat")
27 open(unit=200, file="arquivo-saida-posicoes-t200.dat")
28
29 ! Para cada instante de tempo t, damos um passo aleatório com cada uma das partículas
30 ! Além disso, nos instantes t = 50, 100 e 200 nós salvamos a posição de cada uma (para análise)
31
32 do t=1,t_max
33
34     sum_x = 0.0d0
35     sum_x2 = 0.0d0

```

```

32      do t=1,t_max
33
34          sum_x = 0.0d0
35          sum_x2 = 0.0d0
36
37          do k=1,k_total
38              pos_x(k) = pos_x(k) + step()
39              sum_x = sum_x + pos_x(k)
40              sum_x2 = sum_x2 + pos_x(k)**2
41          end do
42
43          mean_x = sum_x / real(k_total, 8)
44          mean_x2 = sum_x2 / real(k_total, 8)
45
46          write(10, *) t, mean_x, mean_x2
47
48          if(t.eq.50) then
49              call save_vector(50, pos_x, k_total)
50          end if
51
52          if(t.eq.100) then
53              call save_vector(100, pos_x, k_total)
54          end if
55
56          if(t.eq.200) then
57              call save_vector(200, pos_x, k_total)
58          end if
59
60      end do
61
62      close(10)
63      close(50)
64      close(100)
65      close(200)
66
67      stop
68      end
69
70      subroutine null_vector(v, n)
71          integer n, i
72          double precision v(n)
73          do i=1,n
74              v(i) = 0.0d0
75          end do
76          return
77      end
78
79      subroutine save_vector(file_unit, pos, k_total)
80          integer file_unit
81          integer k_total, k
82          double precision pos(k_total)
83          do k=1,k_total
84              write(file_unit, *) pos(k)
85          end do
86          return
87      end
88
89      double precision function step()
90          ! Ajustar o valor do range do step para [-1, 1]
91          step = 2.0d0 * rand() - 1.0d0
92      return
93      end

```

3.2 Passeio aleatório em 2D

3.2.1 Cálculo do valor teórico

Em duas dimensões, temos que $\langle r^2(t) \rangle = \langle x^2(t) + y^2(t) \rangle$. Como x e y são variáveis independentes, vale que

$$\begin{aligned}
\langle r^2(t) \rangle &= \langle x^2(t) + y^2(t) \rangle \\
&= \langle x^2(t) \rangle + \langle y^2(t) \rangle \\
&= \frac{1}{3} + \frac{1}{3} \\
&= \frac{2}{3}
\end{aligned}$$

3.2.2 Resultado das simulações

A figura 13 apresenta a evolução temporal de $\langle r^2(t) \rangle$ em 2D para diferentes tamanhos de conjunto ($k = 50, 100, 200, 500$) em comparação com a curva teórica $y = \frac{2}{3}t$. Observa-se que para $k < 100$, as flutuações estatísticas ainda provocam variações superiores a 10% em relação à reta teórica. A partir de $k \geq 200$, o comportamento do estabiliza e a curva torna-se indistinguível do resultado teórico dentro do limite de precisão de 10%.

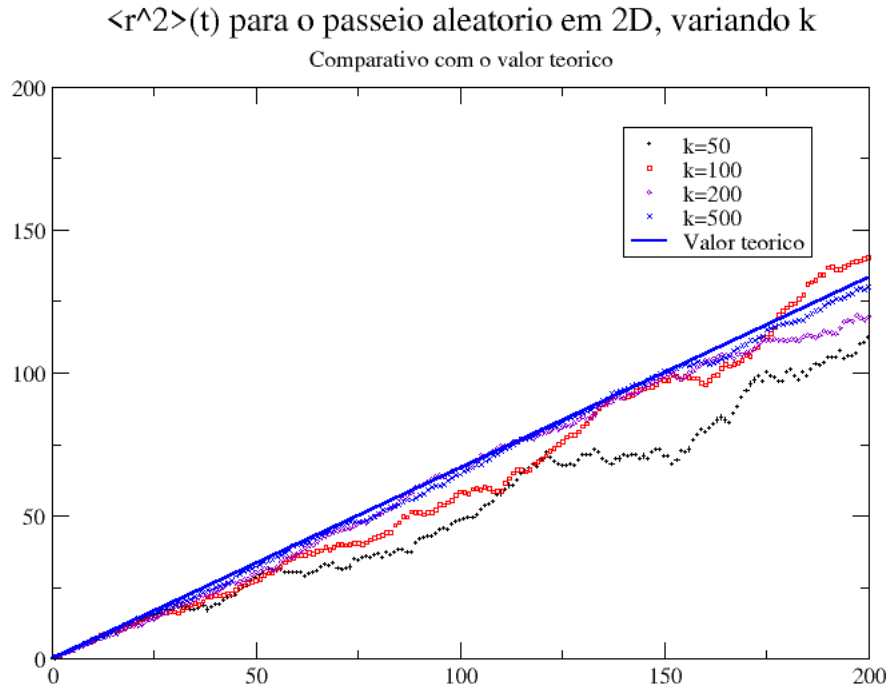


Figura 13: Valor de $\langle r^2 \rangle(t)$, em comparação com o valor esperado de $\langle r^2 \rangle(t) = \frac{2t}{3}$, para $k = 50, 100, 200, 500$

```

implicit none

integer k_total ! Número de seqüências ("passeios", ou "partículas a caminhar")
integer t_max   ! Tempo máximo para a simulação
integer iseed
integer k, t

double precision sum_r2, mean_r2
double precision pos_x(500), pos_y(500) ! Lista com as posição em x,y de cada partícula. O valor é grande para poder variar k_total

double precision step

iseed = 50
k_total = 100
t_max = 200

call srand(iseed)

call null_vector(pos_x, k_total)
call null_vector(pos_y, k_total)

! Arquivo de saída para o gráfico de t vs <r2(t)>
open(unit=10, file="arquivo-saida-r2-tempo.dat")

! Para cada instante de tempo t, damos um passo aleatório com cada uma das partículas, nas duas dimensões
do t=1,t_max

    sum_r2 = 0.0d0

    do k=1,k_total
        pos_x(k) = pos_x(k) + step()
        pos_y(k) = pos_y(k) + step()
        sum_r2 = sum_r2 + (pos_x(k)**2 + pos_y(k)**2)
    end do

    mean_r2 = sum_r2 / real(k_total, 8)

    write(10, *) t, mean_r2

end do

close(10)

stop
end

subroutine null_vector(v, n)
    integer n, i
    double precision v(n)
    do i=1,n
        v(i) = 0.0d0
    end do
return
end

double precision function step()
    ! Ajustar o valor do range do step para [-1, 1]
    step = 2.0d0 * rand() - 1.0d0
return
end

```

Figura 14: Código principal da tarefa 3b, para o passeio aleatório em duas dimensões.

3.3 (Opcional) Cálculo da Entropia e Seta do Tempo